ANÁLISIS CONCEPTUAL DEL CÓDIGO DE SEGUIMIENTO

Robot OSOYOO Mecanum 4WD | Arquitectura lógica, pinout y criterios para una futura implementación independiente de Arduino

Estado del documento: referencia de estudio y diseño inicial. No constituye especificación final ni código destinado a ejecución.

Fuente principal: sketch C/C++ aportado por el usuario, identificado en su encabezado como «Lesson 6 - Object follow Robot car». El manual OSOYOO adjunto se considera únicamente contexto del hardware.

Propósito: separar sensores, lógica de seguimiento, comandos de ruedas y accionamiento físico; reconocer hipótesis, límites y elementos transferibles a una arquitectura nueva.

Qué representa	Qué NO representa
Ejemplo de control reactivo mediante dos sensores IR	Control final del nuevo proyecto
Relación entre comandos lógicos, PWM y direcciones de giro	Selección de microcontrolador o de framework
Mapa de señales utilizado por este fragmento	Verificación del cableado físico instalado
Línea base para diseñar interfaces y probar hipótesis	Control completo de una base Mecanum

Resumen en una frase

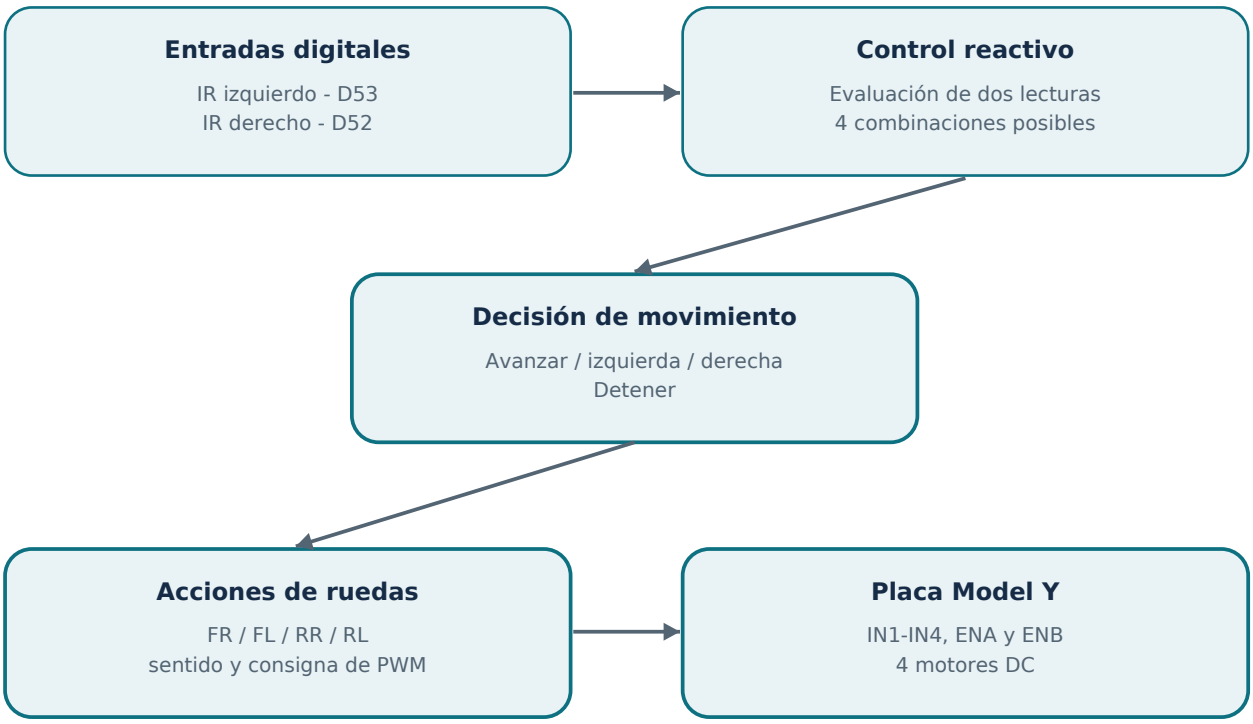
Dos entradas digitales seleccionan una de cuatro acciones; cada acción escribe valores de dirección y PWM en cuatro motores de corriente continua a través de la placa Model Y.

01 Resumen ejecutivo

El algoritmo intenta que el vehículo responda a la presencia de un objeto frente a dos sensores IR. Cuando ambos detectan, avanza; cuando solo uno detecta, gira hacia ese lado; cuando ninguno detecta, se detiene. No hay medición explícita de distancia ni velocidad real.

- Tipo de control: lógica combinacional de estados de sensores + actuadores a lazo abierto.
- Percepción: dos señales digitales con la convención LOW = detección.
- Acción: cuatro comandos básicos del vehículo, implementados mediante ocho rutinas por rueda.
- Actuación: señales digitales de sentido y modulaciones PWM del driver Model Y.
- Tiempo: reevaluación continua mediante la función principal `loop()`.
- Restricción conceptual: no explota el desplazamiento lateral ni diagonal propio de ruedas Mecanum.

Cadena de procesamiento



Flujo lógico conceptual: sensores → decisión → consignas → potencia

Qué puede extraerse para el diseño futuro

Se puede conservar la separación de responsabilidades: adquisición de sensores, evaluación de una política de movimiento, generación de consignas de ruedas y adaptación al hardware. No hace falta conservar llamadas `digitalWrite()`, `analogWrite()`, `pinMode()` ni las funciones de ciclo de vida Arduino.

02 Entradas, salidas y conexionado indicado por el código

El siguiente mapa describe lo que el fragmento declara. La correspondencia de cada motor con la posición física debe comprobarse sobre el robot real, especialmente si se cambió el cableado.

Señales de control de motores

Motor / canal	Dirección A	Dirección B	PWM / EN	Driver
FR · delantero derecho	D22	D24	D9	Model Y M_B · IN1/IN2 · ENA
FL · delantero izquierdo	D26	D28	D10	Model Y M_B · IN3/IN4 · ENB
RR · trasero derecho	D5	D6	D11	Model Y M_A · IN1/IN2 · ENA
RL · trasero izquierdo	D7	D8	D12	Model Y M_A · IN3/IN4 · ENB

Lecturas de sensores

Etiqueta	Pin	Tipo	Uso
LeftSensor	D53	Entrada digital	Sensor IR montado a la izquierda
RightSensor	D52	Entrada digital	Sensor IR montado a la derecha

Resumen de recursos declarados

12 salidas: 8 señales de dirección + 4 señales PWM. 2 entradas: sensores IR. El programa no declara interfaces para ultrasonido, servo, encoders, Bluetooth, WiFi o sensor seguidor de línea de cinco puntos.

Distinción importante: los pines D52/D53 de este ejemplo no equivalen a las entradas A0–A4 utilizadas por el módulo de seguimiento de cinco puntos en otra lección del manual. Son dos configuraciones funcionalmente distintas.

Consignas de velocidad

```
#define SPEED       100
#define TURN_SPEED  100
```

SPEED sí se usa; TURN_SPEED aparece definido pero nunca es utilizado por `auto_following()`. Con la semántica habitual de `analogWrite()` en un Mega, 100 representa aproximadamente 39,2 % del intervalo de consigna 0–255, no 100 RPM ni 100 % de velocidad.

03 Descomposición por bloques funcionales

Bloque	Funciones / elementos	Responsabilidad
A. Configuración	#define SPEED, TURN_SPEED; macros de pines	Define referencias y asignaciones de canales.
B. Actuación individual	FR_fwd/bck, FL_fwd/bck, RR_fwd/bck, RL_fwd/bck	Fija sentido y duty PWM de cada motor.
C. Maniobras	go_advance, go_back, left_turn, right_turn, left_back, right_back	Compone acciones de cuatro ruedas.
D. Detención	stop_Stop	Escribe PWM = 0 en los cuatro canales.
E. Inicialización	init_GPIO, setup	Configura pines e impone detención inicial.
F. Percepción y decisión	auto_following	Lee dos entradas y selecciona una maniobra.
G. Ciclo de ejecución	loop	Repite la toma de decisión.

B. Control de motor individual

```
void FR_fwd(int speed) {
    digitalWrite(RightMotorDirPin1, LOW);
    digitalWrite(RightMotorDirPin2, HIGH);
    analogWrite(speedPinR, speed);
}
```

Cada motor usa dos señales de sentido (A/B) y una consigna PWM. Las ocho rutinas son variantes de la misma estructura. La etiqueta «forward» depende del cableado y de la orientación física: el nombre de la función por sí solo no verifica que la rueda avance.

C. Composición de maniobras

```
void go_advance(int speed) {
    RL_fwd(speed); RR_fwd(speed);
    FR_fwd(speed); FL_fwd(speed);
}
```

La capa de maniobras evita que la lógica de sensores tenga que conocer los pines. Es el principal patrón arquitectónico reutilizable, pero la mezcla de velocidades debe rediseñarse si se desea cinemática Mecanum general.

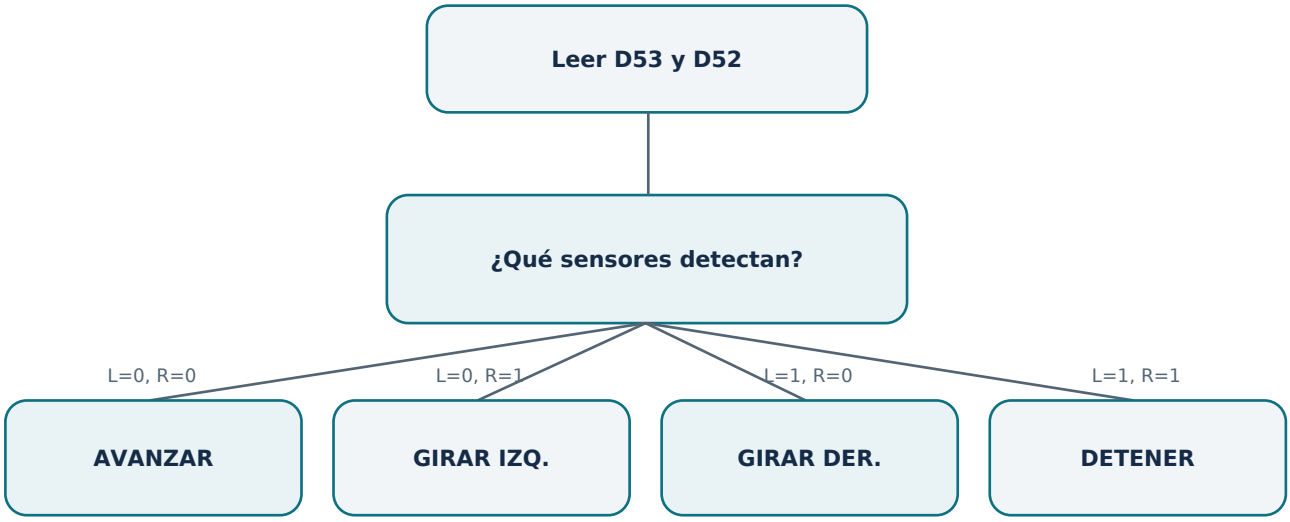
D–G. Secuencia de ejecución

`setup()` ejecuta `init_GPIO()` una vez; esta función configura entradas/salidas y detiene motores. `loop()` llama continuamente a `auto_following()`. En un entorno nuevo, estas responsabilidades pueden corresponder a inicialización de plataforma y tareas periódicas.

04 Tabla de verdad del comportamiento

La tabla se expresa por el cableado nominal de los sensores, no por los nombres invertidos de variables locales.

Izquierdo D53	Derecho D52	Condición interpretada	Función seleccionada
LOW	LOW	Ambos detectan	go_advance(SPEED)
LOW	HIGH	Solo izquierdo detecta	left_turn(SPEED)
HIGH	LOW	Solo derecho detecta	right_turn(SPEED)
HIGH	HIGH	Ninguno detecta	stop_Stop()



0 = LOW = detección asumida por el sketch. 1 = HIGH = no detección.

Anomalía de nombres de las variables

```
int IRvalueLeft  = digitalRead(RightSensor);
int IRvalueRight = digitalRead(LeftSensor);
```

La variable llamada «Left» contiene en realidad la lectura del sensor derecho, y viceversa. Por eso la asociación entre cada rama y su comentario debe analizarse desde el pin físico. Si en una refactorización se corrigen los nombres pero no las condiciones, se podría invertir la decisión de giro.

Ejemplo de secuencia conceptual

Objeto detectado por ambos sensores → avanzar. El objeto se desplaza hacia el lado izquierdo y únicamente el IR izquierdo conserva la detección → girar hacia la izquierda. Si no hay detección → detener los cuatro PWM. Esta secuencia es una expectativa lógica y no garantiza que el robot siga al objeto sin perderlo.

05 Matriz de actuación de ruedas

FR = delantera derecha; FL = delantera izquierda; RR = trasera derecha; RL = trasera izquierda. En la tabla, «+S» es sentido forward con PWM = SPEED; «−S» es backward con PWM = SPEED; «0» indica PWM nulo.

Maniobra	FL	FR	RL	RR	Observación
go_advance	+S	+S	+S	+S	Las cuatro ruedas reciben avance.
go_back	−S	−S	−S	−S	Las cuatro ruedas reciben retroceso.
left_turn	0	+S	0	+S	Lado izquierdo sin PWM; derecho avanza.
right_turn	+S	0	+S	0	Lado derecho sin PWM; izquierdo avanza.
left_back	0	−S	0	−S	Lado derecho retrocede; no se llama.
right_back	−S	0	−S	0	Lado izquierdo retrocede; no se llama.
stop_Stop	0	0	0	0	PWM = 0; dirección anterior no se cambia.

Interpretación física de los giros

Las maniobras etiquetadas «turn» no ordenan contrarrotación de ruedas: dejan un lado con PWM = 0 y hacen avanzar el otro. Por lo tanto se parecen a un giro por diferencia de velocidades entre lados, potencialmente combinado con traslación. No equivalen automáticamente a una rotación pura sobre el centro.

Semántica de detención

Detener por PWM = 0 no implica necesariamente un frenado eléctrico activo. Que el motor quede en rueda libre o frenado depende del circuito del puente H. La política de parada segura debe definirse desde el driver real, no inferirse del nombre `stop_Stop()`.

Sentido lógico de un canal

Control	IN1	IN2	PWM	Interpretación en este sketch
fwd	LOW	HIGH	speed	Sentido asignado como avance
bck	HIGH	LOW	speed	Sentido asignado como retroceso
stop	sin cambios	sin cambios	0	Sin accionamiento PWM

06 Secuencia y comportamiento temporal

Desde el arranque a la ejecución repetitiva

1. Inicio del entorno: ejecuta `setup()` → `init_GPIO()`.
2. Se configuran ocho pines de dirección, cuatro PWM y las dos entradas digitales.
3. Se llama a `stop_Stop()` para comenzar con PWM nulo.
4. En cada repetición de `loop()`, se leen las dos entradas.
5. Se evalúa una única combinación y se actualizan los cuatro motores.
6. Se vuelve inmediatamente al paso 4, sin temporizador ni período fijo declarado.

Qué no está definido

- No existe muestreo periódico explícito, filtrado ni histéresis para señales IR inestables.
- No hay tiempo mínimo de maniobra, ni aceleración/rampa en los PWM.
- No hay sincronización especial entre cambios de dirección y actualización del PWM.
- No hay control del tiempo transcurrido desde la última detección.
- No se comprueba una distancia mínima ni se establecen límites de aproximación.
- No se usa realimentación de encoder, corriente o velocidad.

Comportamiento ante situaciones relevantes

Situación	Respuesta actual	Consecuencia de diseño
Objetivo ausente	Stop por doble HIGH	No hay búsqueda activa para reacquirirlo.
Objetivo muy próximo	Puede seguir avanzando con doble LOW	Riesgo de contacto; faltan límites de proximidad.
Detección intermitente	Cambio inmediato de acción	Puede oscilar o moverse a tirones.
Batería cae	Mismo PWM numérico	La velocidad/torque reales pueden cambiar.
Sensor desconectado	No se identifica fallo	Puede confundirse con estado HIGH o LOW.

07 Inconsistencias, errores potenciales y limitaciones

Hallazgo	Evidencia en el sketch	Impacto
Variables de sensores cruzadas	IRvalueLeft lee RightSensor; IRvalueRight lee LeftSensor	Al renombrar sin revisar condiciones pueden invertirse giros.
Constante sin uso	TURN_SPEED está declarada; se llama left_turn(SPEED)	No existe ajuste independiente del giro.
Funciones sin uso	go_back, left_back y right_back nunca se llaman desde auto_following	No forman parte del comportamiento ejecutado.
Comentarios contradictorios	go_advance(SPEED) incluye comentario «Turn left»	Los comentarios no son evidencia de la acción.
Lógica binaria limitada	Solo LOW/HIGH por cada sensor	No conoce distancia, posición angular ni tamaño del objeto.
Ausencia de parada por cercanía	Doble LOW siempre avanza	Puede acercarse indefinidamente al objetivo.
Cinemática Mecanum incompleta	No hay traslación lateral/diagonal	No ofrece control omnidireccional general.
Consigna sin validación	speed se pasa a analogWrite sin comprobación propia	Depende del rango válido de la plataforma.
Sin gestión de fallos	No se comprueban hardware, alimentación ni sensores	Difícil diagnosticar y garantizar seguridad.
Sin watchdog de seguridad	loop repite sin mecanismo independiente	El control del sistema no garantiza respuesta ante cuelgue.

Tres confusiones que conviene evitar

- Movimiento vs. velocidad real: 100 es una consigna PWM, no una magnitud cinemática medida.
- «Girar» vs. «rotar en el lugar»: en estas funciones, el lado inactivo no rota impulsado hacia atrás.
- Vehículo Mecanum vs. control Mecanum: utilizar ruedas Mecanum no implica que el código implemente el modelo cinemático de cuatro grados de control de rueda.

08 Arquitectura propuesta independiente del framework

La siguiente separación no aparece implementada literalmente en el código original. Es una propuesta de diseño derivada de las responsabilidades observadas, para utilizar en C/C++ con la plataforma y el entorno de ejecución que finalmente se elijan.

Capa futura	Responsabilidad	Equivalente conceptual del ejemplo
HAL / Board Support	GPIO, temporizadores PWM, interrupciones y seguridad de salida	digitalRead, digitalWrite, analogWrite, pinMode
Sensores	Obtener y validar detecciones normalizadas	Lectura D52/D53 de auto_following
Actuadores	Conducir cada motor: dirección, magnitud, stop	FR_fwd/bck ... RL_fwd/bck; stop_Stop
Movimiento del chasis	Traducir velocidades o maniobras a consignas individuales	go_advance, left_turn, right_turn ...
Comportamiento	Decidir qué quiere hacer el robot a partir del objetivo	if/else de auto_following
Supervisión	Timeout, límites, estado de fallos y parada segura	No existe en el ejemplo

Contrato de datos mínimo propuesto

```
Entradas: {detecta_izquierda, detecta_derecha, valido}
Salida lógica: {AVANZAR, GIRAR_IZQ, GIRAR_DER, PARAR}
Salida motriz: {motor_FL, motor_FR, motor_RL, motor_RR}
Cada motor: {sentido, magnitud_normalizada}
```

Estos nombres son expresiones abstractas para documentar el contrato funcional. No son una API definida ni sintaxis obligatoria de la nueva implementación.

Pseudocódigo independiente de plataforma

```
observaciones = leer_y_validar_sensores()
if observaciones.no_validas:
    orden = PARADA_SEGURA
else:
    orden = decidir_seguimiento(observaciones)
consignas = traducir_orden_a_motores(orden)
aplicar_consignas(consignas)
```

Este pseudocódigo muestra una arquitectura posible, no el algoritmo exacto existente. Introduce explícitamente un estado de validación y supervisión que el sketch no tiene.

09 Qué conservar, qué reemplazar y qué verificar

Elemento	Tratamiento recomendado	Motivo
Matriz de cuatro estados IR	Conservar como primera hipótesis de comportamiento	Ofrece una referencia sencilla para pruebas iniciales.
Separación sensor → decisión → motores	Conservar como criterio de arquitectura	Permite desacoplar lógica y hardware.
Mapa de pines	Conservar como referencia documental; verificar	Podría variar el cableado real o la placa destino.
Polos HIGH/LOW	Calibrar y normalizar en capa sensora	La polaridad depende del sensor empleado.
Consigna SPEED = 100	No trasladar como velocidad física	Solo corresponde a un valor nominal PWM.
Rutinas digitalWrite / analogWrite	Reemplazar completamente	Están ligadas al entorno Arduino.
Control de motores	Rediseñar como interfaz de driver	Permite otro microcontrolador o periférico.
Lógica de maniobra Mecanum	Ampliar o reemplazar según objetivo	El ejemplo no cubre todos los movimientos.
Estrategia ante pérdida del objeto	Definir nueva política	La única respuesta actual es parar.
Medidas de seguridad	Incorporar explícitamente	El sketch no previene acercamiento excesivo ni fallos.

Parámetros que este fragmento no aporta

- Modelo eléctrico exacto y características térmicas del puente H Model Y.
- Tipo específico de los dos sensores IR, ajuste, ángulo y rango efectivo de detección.
- Relación de reducción, RPM máximas, corriente y constante de par de los motores.
- Radio efectivo de las ruedas, distancias entre centros, masa e inercia del chasis.
- Frecuencia real de PWM, respuesta del motor a cada duty y velocidad a distintas tensiones.
- Estado físico de frenado con PWM cero en la placa instalada.

10 Plan mínimo para validar el concepto

La finalidad de estas pruebas es validar supuestos antes de desarrollar un control más complejo. Deben realizarse inicialmente con el chasis levantado o con las ruedas sin contacto, con limitación de potencia y una detención accesible.

Prueba	Procedimiento	Criterio de aceptación
P01 · Identificación de rueda	Accionar de forma individual FR, FL, RR y RL a baja consigna	Coincidencia entre rueda y canal asignado.
P02 · Sentido	Aplicar polaridades fwd/bck a baja consigna	Avance/retroceso coherentes con cada etiqueta.
P03 · Parada	Ordenar PWM nulo después de cada sentido	Verificar si frena o queda libre, sin inferencias.
P04 · IR izquierdo	Presentar objeto solo ante el sensor D53	Lectura LOW si es activo en bajo.
P05 · IR derecho	Presentar objeto solo ante el sensor D52	Lectura LOW si es activo en bajo.
P06 · Tabla de verdad	Recrear las cuatro combinaciones binarias	Coincidencia exacta con la tabla de la sección 04.
P07 · Pérdida de objetivo	Retirar ambos estímulos IR	Consigna de parada y ausencia de maniobra persistente.
P08 · Cercanía	Aproximar objeto sin mover el vehículo	Determinar si hace falta un sensor de distancia.
P09 · Sensibilidad	Repetir con distintos materiales y luz ambiental	Identificar falsos positivos y variaciones.
P10 · Seguridad	Interrumpir señales / simular fallo	Definir la reacción segura faltante.

Criterios de migración del proyecto

Antes de trasladar la lógica, conviene fijar una convención única de orientación del robot, nombrar inequívocamente los sensores, identificar las cuatro ruedas, definir valores normalizados de consigna y diferenciar parada solicitada de parada de seguridad.

11 Registro de hechos, hipótesis y propuestas

Categoría	Enunciado	Grado de confianza
Hecho del código	Se configuran 12 salidas y 2 entradas, D52/D53 para sensores.	Alto: lectura directa del fragmento.
Hecho del código	LOW/LOW genera avanzar; HIGH/HIGH genera detener.	Alto: lectura directa del fragmento.
Hecho del código	La decisión se repite y no existen lecturas de velocidad.	Alto: lectura directa del fragmento.
Hipótesis operativa	LOW corresponde a un objeto realmente detectado.	A confirmar con la electrónica.
Hipótesis operativa	Los nombres de motor se corresponden con posiciones reales.	A confirmar mediante pruebas.
Hipótesis operativa	El driver permite comportamiento seguro con PWM = 0.	No establecido.
Propuesta	Abstraer driver, controladores y adquisición sensora.	Decisión de arquitectura futura.
Propuesta	Agregar validez de sensor, timeout y límites de proximidad.	Decisión de diseño y seguridad futura.
Propuesta	Añadir cinemática Mecanum para movimientos holonómicos.	Solo si el nuevo objetivo funcional lo requiere.

Alcance del manual OSOYOO frente a este sketch

La documentación de hardware original describe la plataforma Mecanum, Model Y y el esquema de pines de los motores. Este sketch añade un caso de seguimiento con dos entradas IR digitales. No se debe confundir con las actividades de evasión ultrasónica ni de seguimiento de línea con cinco sensores que aparecen en el manual.

Conclusión técnica

La principal utilidad de este fragmento es la forma de particionar un sistema reactivo de locomoción: entradas de sensores → decisión de alto nivel → composición de maniobras → control individual de actuadores. Su valor es conceptual y didáctico; no justifica conservar la dependencia de Arduino ni reproducir ciegamente sus giros, números PWM y condiciones de seguridad.

Resultado esperado para la siguiente fase: una especificación de interfaces independiente de la plataforma, validada contra el pinout real, que permita evolucionar hacia sensores calibrados, control más preciso de motores y, si corresponde, cinemática Mecanum.

Fuente primaria: código suministrado en la conversación. Fuente secundaria de contexto: «ManualOsoyoo.pdf», lecciones de ensamblaje, conexionado y control del vehículo. Los aspectos propuestos se identifican explícitamente como propuestas, no como funciones presentes en el sketch.